

Informe

Arquitectura Resiliente

NAC-PTITARI23-00000002

COORDINACIÓN DE GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA

Departamento de Planificación TI y Gestión de la Información - Datos

Dirección Nacional de Tecnología

Marzo 2023

Versión: 1.0.0

Firmas de Responsabilidad

Dirección Nacional de Tecnología		Fecha: 2023-03-15	VERSIÓN 1.0.0	PÁGINAS 32
NOMBRE Y CARGO			FIRMA	
REVISADO Y APROBADO POR:	Ing. Paulina Correa A. JEFE NACIONAL DE DEPARTAMENTO DE PLANIFICACIÓN TI Y GESTIÓN DE INFORMACIÓN- DATOS		 Firmado electrónicamente por: PAULINA ELIZABETH CORREA AGUAYO	
ELABORADO POR:	Ing. Edison Cayambe Ch. COORDINADOR NACIONAL DE GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA		 Firmado electrónicamente por: EDISON ESTUARDO CAYAMBE CHICAIZA	

Tabla de contenidos

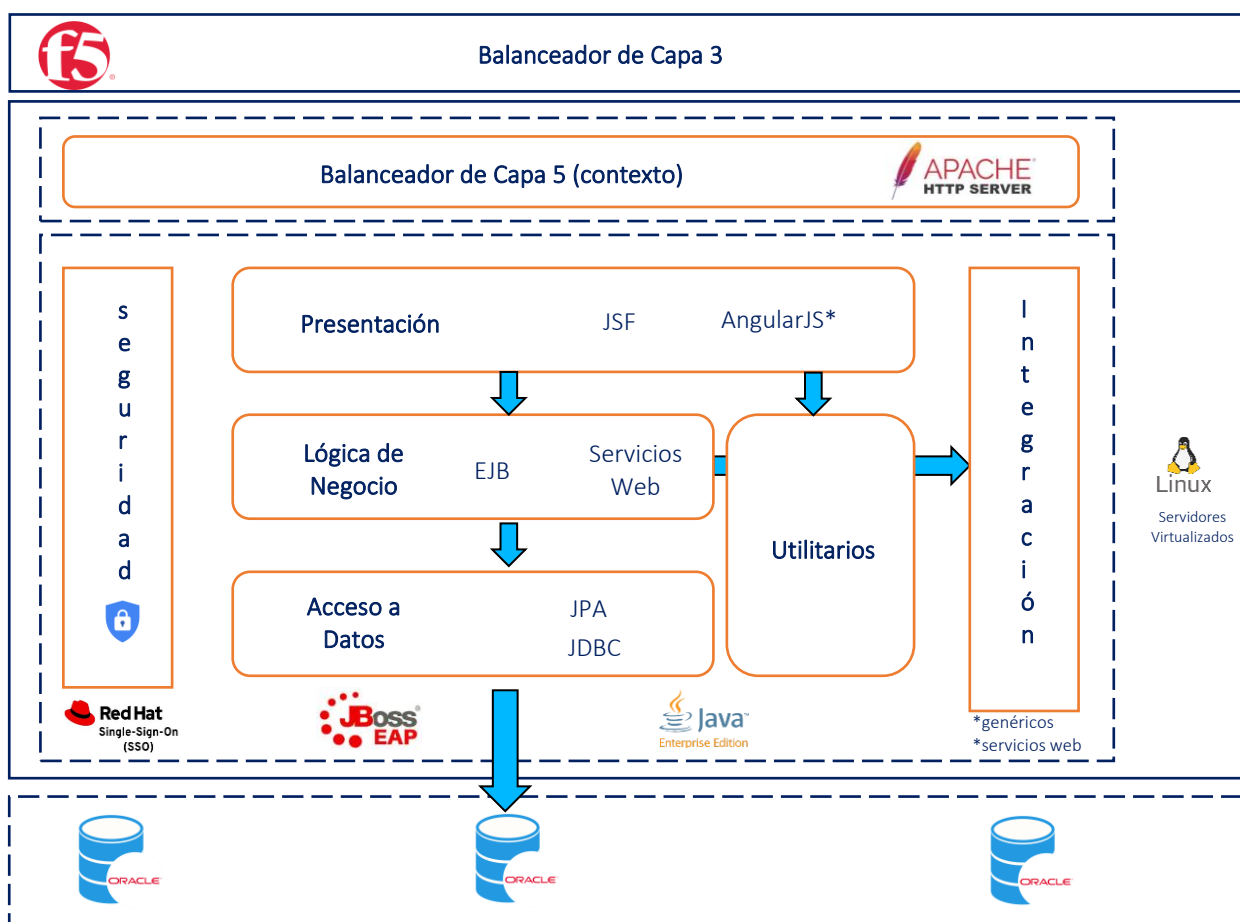
1.	Antecedentes.....	3
2.	Alcance	4
3.	Metodología	4
4.	Plan de trabajo	6
5.	Equipo de trabajo	8
6.	Hitos cumplidos	8
7.	Conclusiones.....	30
8.	Recomendaciones.....	30
9.	Glosario	31

1. Antecedentes

Las aplicaciones del Servicio de Rentas Internas están desarrolladas bajo la definición de una arquitectura por capas con la especificación Java EE, las cuales están desplegadas sobre servidores de aplicaciones JBoss EAP. Bajo esta arquitectura, se ha evidenciado a lo largo del tiempo algunas dificultades entre ellas:

- Dependency hell:** Existen aplicaciones que por dependencias transitivas llegan a requerir hasta 9 artefactos para su despliegue y no necesariamente son usadas dentro de la lógica del giro de negocio de la aplicación.
- Escalamiento ineficiente:** Si un artefacto de una aplicación requiere mayor capacidad, la arquitectura actual obliga a escalar la instancia completa de la aplicación.
- Down Time:** Cuando se requiere dar mantenimiento a un artefacto de una aplicación, se genera en toda la aplicación un tiempo de inactividad.
- Fallos totales:** Cuando un artefacto de una aplicación falla, degrada por completo el funcionamiento de la aplicación.
- Alto acoplamiento entre aplicaciones:** Existen artefactos transversales que cuando son actualizados, es necesario que todas las aplicaciones sean actualizadas, lo cual genera un alto acoplamiento entre estas.

A continuación, se detalla los componentes de la arquitectura actual:



Para el 01/08/2022 arranca la iniciativa que va a permitir definir para el Servicio de Rentas Internas, una nueva arquitectura de aplicaciones denominada: “**Arquitectura Resiliente**” cuyo objetivo es proveer los siguientes beneficios:

- a) Aplicaciones modulares con componentes independientes
- b) Escalamiento eficiente
- c) Mantenimientos con bajo Down Time
- d) Aislamiento de fallos
- e) Bajo acoplamiento entre aplicaciones

Esta iniciativa, será implementada inicialmente como piloto en el proyecto base intranet.

2. Alcance

El presente informe describe como se abordó y ejecutó la iniciativa “**Arquitectura Resiliente**” de acuerdo con el siguiente detalle:

- 1. Metodología empleada
- 2. Plan de trabajo establecido
- 3. Equipo de trabajo
- 4. Hitos cumplidos
- 5. Conclusiones
- 6. Recomendaciones

3. Metodología

Se plantea ejecutar la iniciativa bajo una metodología ágil que permita obtener entregables parciales de valor para la nueva arquitectura, dichos entregables se definieron como hitos a cumplir y que se listan a continuación:

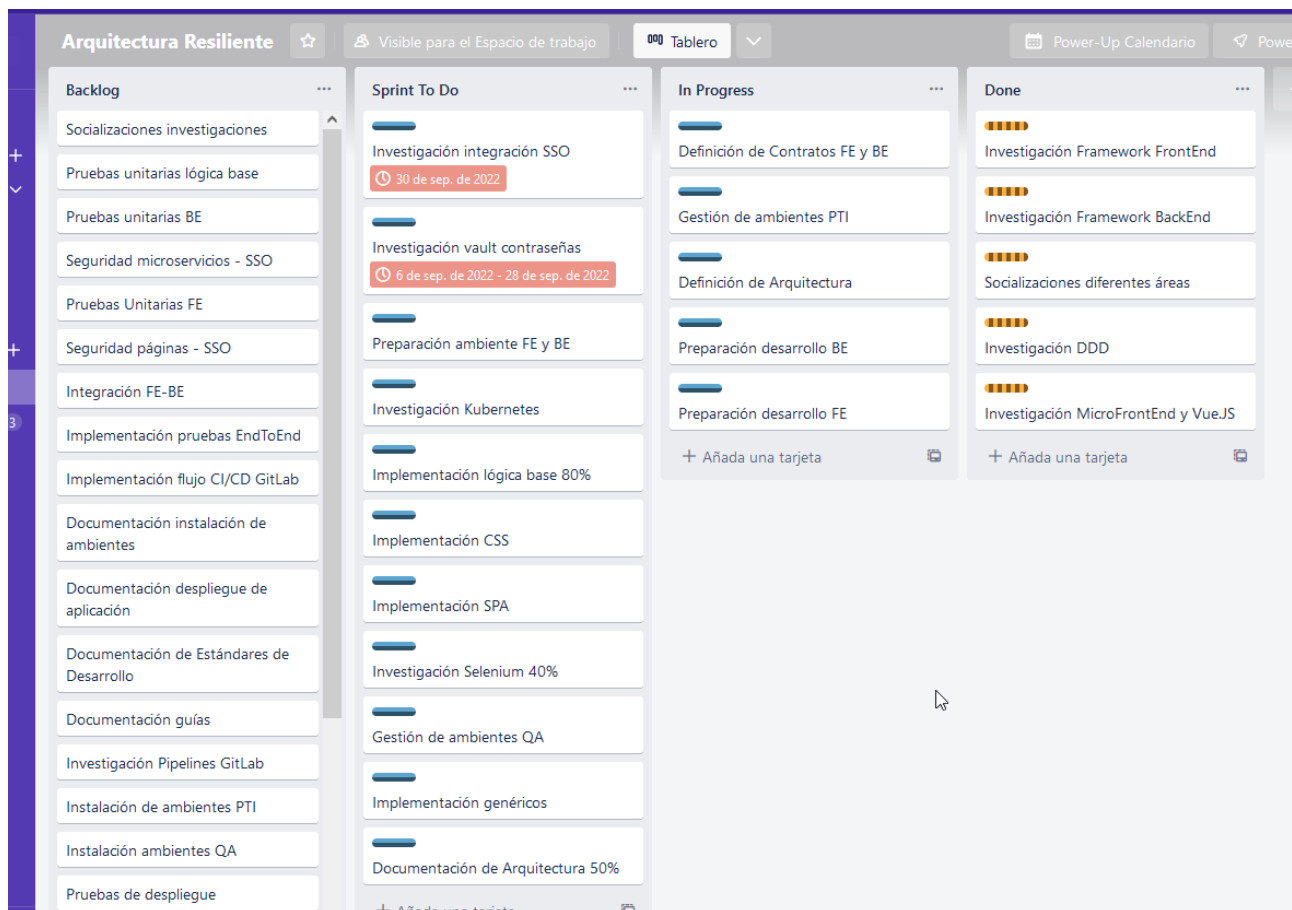
- 1. **INVESTIGACIÓN:** Investigaciones de tecnologías emergentes que son candidatas para usar dentro de la definición de la nueva arquitectura.
- 2. **PRUEBA DE CONCEPTO:** Pruebas de concepto de las tecnologías emergentes candidatas.
- 3. **ARQUITECTURA & DISEÑO:** Definición de Arquitectura y Diseño.
- 4. **DEVELOPMENT BE:** Implementación de API's Back End del proyecto base intranet.
- 5. **DEVELOPMENT FE:** Implementación vistas Front End de proyecto base intranet.
- 6. **SEGURIDAD:** Implementación de autenticación mediante el componente Red Hat Single Sign-On en las API's Back End y vistas Front End.
- 7. **PREPARACIÓN AMBIENTE:** Instalación de ambientes.
- 8. **DESPLIEGUE:** Despliegue de aplicación piloto en el laboratorio del cluster de kubernetes

Dichos hitos se descomponen en un conjunto de tareas que conformaron el Product Backlog de la iniciativa:

Hito	Product Backlog
ARQUITECTURA & DISEÑO	Definición de Contratos BE
	Definición de Arquitectura
DESPLIEGUE	Contenerización API's
	Contenerización App Base
	Configuración Push
	Configuración Pull
	Manifest Ingress Controller
	Manifest Ingress
	Manifest POD's API's
	Manifest POD's App Base
	Manifest Service API's
	Manifest Service App Base
	Configuración Load Balancer (Apache)
DEVELOPMENT BE	Preparación ambiente local BE
	Implementación API autenticación
	Implementación API catalogo
	Implementación API parámetro
	Implementación API autorización
	Contrato API parámetro
DEVELOPMENT FE	Implementación CSS
	Preparación ambiente local FE y BE
	Implementación template base
	Implementación CRUD parámetro
	Implementación Cache Pinia
	Integración FE-BE
INVESTIGACIÓN	Investigación Selenium
	Investigación Vue.JS
	Investigación Quarkus
	Investigación Secret kubernetes
	Investigación Registry Privado
	Investigación DDD
PREPARACIÓN AMBIENTE	Gestión de laboratorio PTI
	Gestión permisos firewall
	Instalación cluster Kubernetes
	Instalación Registry privado
	Instalación GitLab
PRUEBA DE CONCEPTO	Frameworks BackEnd
	Contenerización de App Vue y Quarkus
	Socializaciones
	Frameworks FronEnd

SEGURIDAD	Seguridad API's - SSO
	Seguridad páginas web - SSO

Dicho Product Backlog fue administrado mediante la herramienta Trello:



4. Plan de trabajo

La definición de la arquitectura resiliente debe ser implementada en una aplicación, por lo cual se elige el proyecto base intranet como piloto a aplicar la nueva arquitectura.

A continuación, se detalla el plan de trabajo ejecutado:

Hito	Tarea	Fecha inicio	Fecha fin	Recurso
ARQUITECTURA & DISEÑO	Definición de Contratos BE	25/7/2022	29/7/2022	Edison Cayambe
	Definición de Arquitectura	1/8/2022	5/8/2022	Edison Cayambe
DESPLIEGUE	Contenerización API's	17/10/2022	28/10/2022	Christian Cabezas
	Contenerización App Base	19/12/2022	23/12/2022	Edison Cayambe
	Configuración Push	31/10/2022	11/11/2022	Christian Cabezas

	Configuración Pull	7/11/2022	11/11/2022	Christian Arreaga
	Manifest Ingress Controller	22/11/2022	2/12/2022	Edison Cayambe
	Manifest Ingress	5/12/2022	16/12/2022	Edison Cayambe
	Manifest POD's API's	23/11/2022	30/11/2022	Christian Arreaga
	Manifest POD's App Base	26/12/2022	30/12/2022	Christian Arreaga
	Manifest Service API's	1/12/2022	9/12/2022	Christian Arreaga
	Manifest Service App Base	19/12/2022	23/12/2022	Christian Arreaga
	Configuración Load Balancer (Apache)	12/12/2022	16/12/2022	Christian Arreaga
DEVELOPMENT BE	Preparación ambiente local BE	5/8/2022	12/8/2022	Christian Arreaga, Christian Cabezas
	Implementación API autenticación	15/8/2022	26/8/2022	Christian Arreaga
	Implementación API catalogo	15/8/2022	26/8/2022	Christian Cabezas
	Implementación API parámetro	14/11/2022	25/11/2022	Christian Cabezas
	Implementación API autorización	29/8/2022	9/9/2022	Christian Cabezas
	Contrato API parámetro	24/10/2022	28/10/2022	Edison Cayambe
DEVELOPMENT FE	Implementación CSS	1/8/2022	23/12/2022	Liliana Rodríguez
	Preparación ambiente local FE y BE	25/7/2022	29/7/2022	Carmen Muñoz, Liliana Rodríguez
	Implementación template base	1/8/2022	19/8/2022	Carmen Muñoz
	Implementación CRUD parámetro	14/11/2022	16/12/2022	Carmen Muñoz
	Implementación Cache Pinia	6/9/2022	23/9/2022	Carmen Muñoz
	Integración FE-BE	26/9/2022	14/10/2022	Carmen Muñoz, Christian Cabezas
INVESTIGACIÓN	Investigación Selenium	15/8/2022	25/11/2022	Oscar Celin
	Investigación Vue.JS	11/7/2022	22/7/2022	Carmen Muñoz
	Investigación Quarkus	27/6/2022	1/7/2022	Edison Cayambe
	Investigación Secret kubernetes	28/11/2022	23/12/2022	Christian Cabezas
	Investigación Registry Privado	3/10/2022	14/10/2022	Christian Arreaga
	Investigación DDD	18/7/2022	22/7/2022	Edison Cayambe
PREPARACIÓN AMBIENTE	Gestión de laboratorio PTI	25/7/2022	12/8/2022	Benito Cuenca
	Gestión permisos firewall	15/8/2022	19/8/2022	Benito Cuenca
	Instalación cluster Kubernetes	29/8/2022	30/9/2022	Christian Arreaga
	Instalación Registry privado	17/10/2022	21/10/2022	Christian Arreaga
	Instalación GitLab	24/10/2022	4/11/2022	Christian Arreaga
PRUEBA DE CONCEPTO	Frameworks BackEnd	13/6/2022	24/6/2022	Edison Cayambe
	Contenerización de App Vue y Quarkus	4/7/2022	8/7/2022	Edison Cayambe
	Socializaciones	11/7/2022	15/7/2022	Edison Cayambe, Carmen Muñoz
	Frameworks FronEnd	13/6/2022	8/7/2022	Carmen Muñoz
SEGURIDAD	Seguridad API's - SSO	12/9/2022	23/9/2022	Christian Cabezas
	Seguridad páginas web - SSO	17/10/2022	11/11/2022	Carmen Muñoz

5. Equipo de trabajo

Con el personal de la Dirección de Tecnología se conforma un equipo con una asignación parcial de acuerdo con el siguiente detalle:

Recurso	Área	Asignación diaria
Carmen Muñoz	Planificación TI	40%
Christian Arreaga	Planificación TI	20%
Liliana Rodriguez	Planificación TI	20%
Oscar Celin	Planificación TI	30%
Benito Cuenca	Planificación TI	20%
Edison Cayambe	Planificación TI	30%
Christian Cabezas	Desarrollo	30%

6. Hitos cumplidos

1. INVESTIGACIÓN

Hito cuyo alcance fue realizar investigaciones para la adopción de lo siguiente:

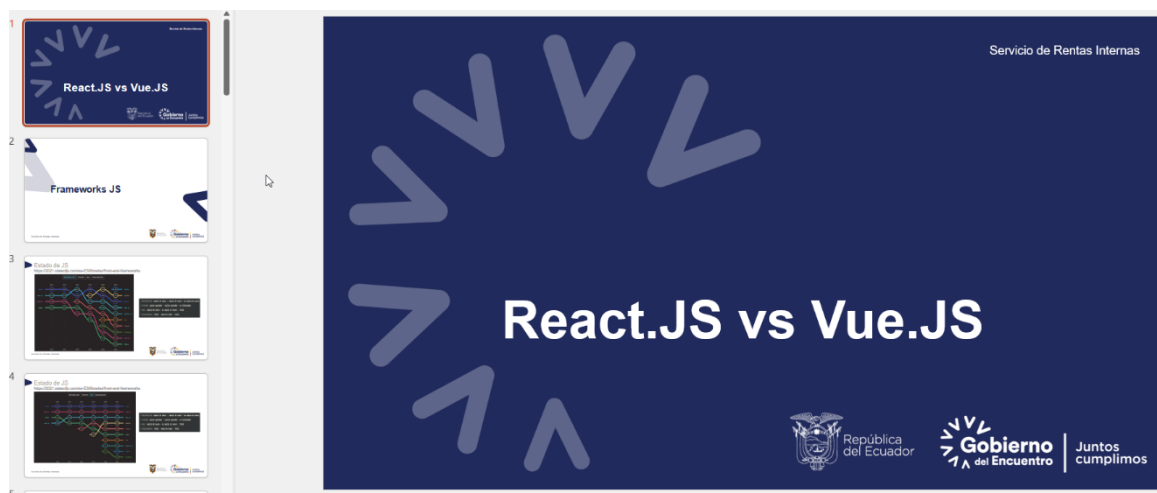
1.1. Quarkus:

Investigación que fue ejecutada y socializada mediante presentación (QuarkusVsSpringBoot.pdf) al equipo de Planificación TI donde se plantearon ventajas y desventajas de cada alternativa planteada:



1.2. Vue.JS:

Investigación que fue ejecutada y socializada mediante presentación (ReactJsVsVueJs.pdf) al equipo de Planificación TI, donde se plantearon ventajas y desventajas de cada alternativa planteada:



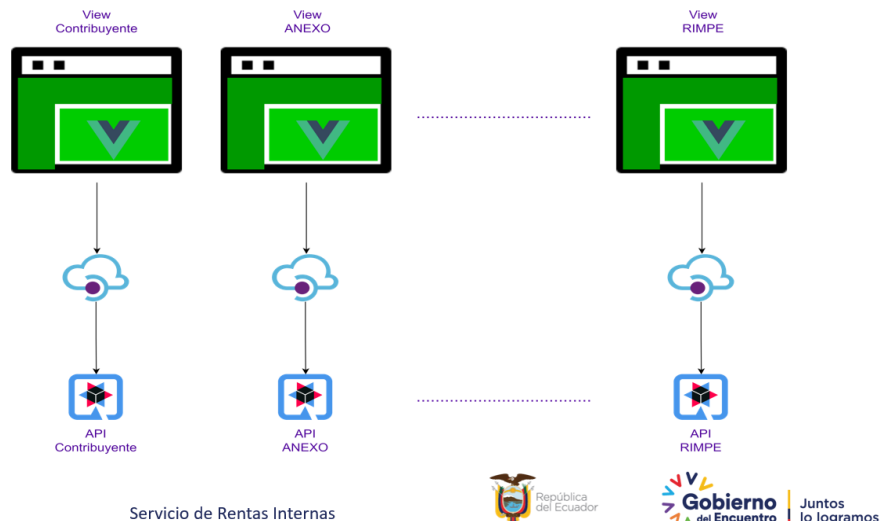
1.3. Domain Driven Design (DDD):

Investigación que fue ejecutada y socializada mediante presentación (ArquitecturaContratos.pdf) al equipo de Planificación TI, donde se determina que la nueva arquitectura tiene un enfoque vertical apalancados en dominios definidos por DDD:



La visión de la nueva arquitectura es crear compoenetes tanto FrontEnd como BackEnd por cada dominio identificado:

Nueva Arquitectura



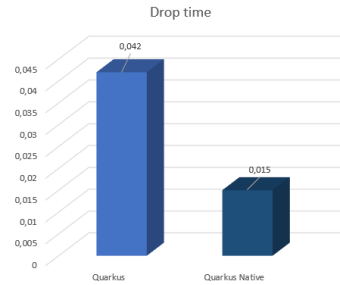
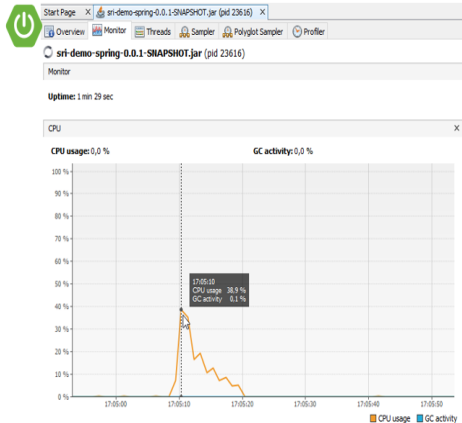
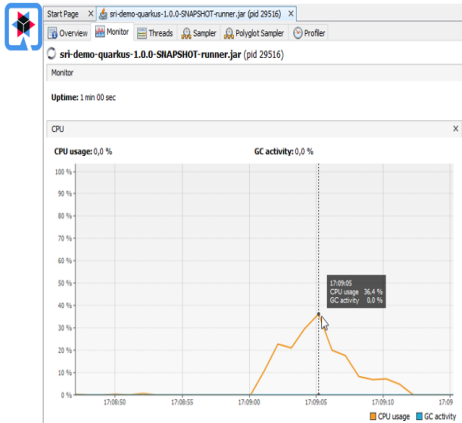
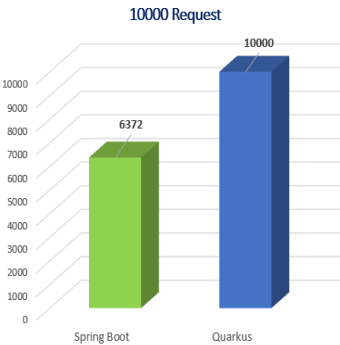
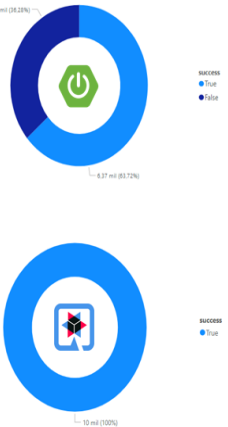
2. PRUEBA DE CONCEPTO

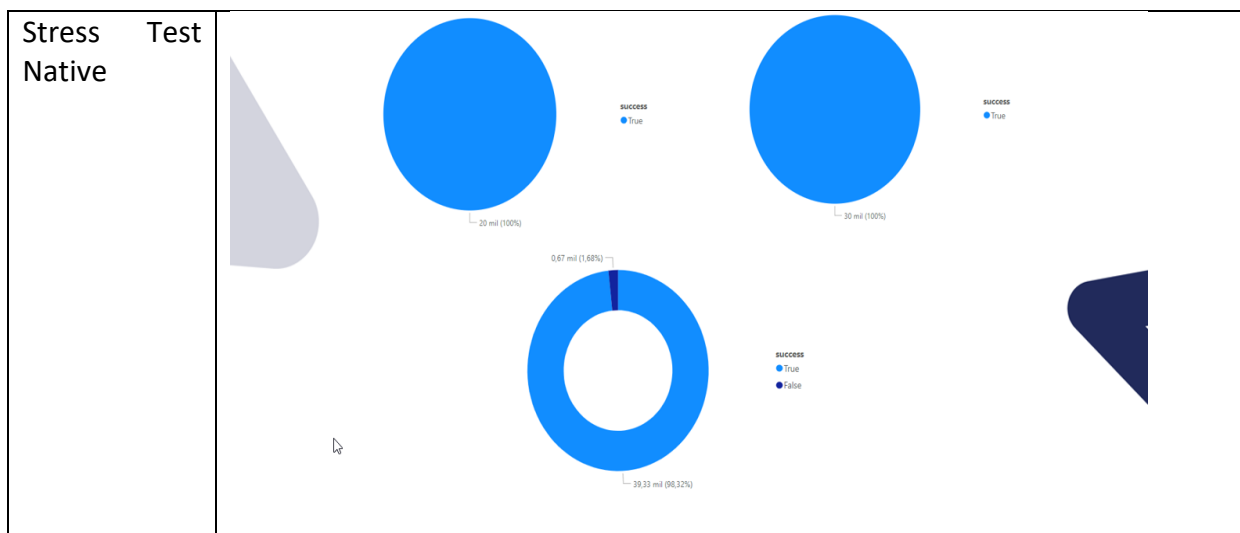
Quarkus

Ejecutada mediante una comparativa entre Quarkus y Spring Boot misma que fue socializada mediante presentación al equipo de Planificación TI:

Dicha comparación se realizó en los siguientes criterios:

Stereotype	sri-demo-spring [boot] [devtools] - src/test/java - src/main/java - com.sri.gob.ec.demo - com.sri.gob.ec.demo.controller - com.sri.gob.ec.demo.modelo - com.sri.gob.ec.demo.repository - com.sri.gob.ec.demo.service - com.sri.gob.ec.demo.service.to - src/main/resources - JRE System Library [jre] - Referenced Libraries - src - target - HELP.md - mvnw - mvnw.cmd - pom.xml sri-demo-quarkus - src/test/java - src/main/java - com.sri.gob.ec - com.sri.gob.ec.controller - com.sri.gob.ec.modelo - com.sri.gob.ec.repository - com.sri.gob.ec.service - com.sri.gob.ec.service.to - src/main/resources - JRE System Library [jre] - Referenced Libraries - src - target - mvnw - mvnw.cmd - pom.xml
Code	<pre> @RestController @RequestMapping("/personas") public class PersonaControllerImpl { @Autowired private IPersonaService personaService; @GetMapping(path =("/{id}") public ResponseEntity<PersonaTo> consultar(@PathVariable("id") Integer id) { return ResponseEntity.ok(this.personaService.buscarPorId(id)); } @PostMapping public ResponseEntity<String> insertar(@RequestBody PersonaTo cto) { this.personaService.insertar(cto); return ResponseEntity.ok("Persona insertada con éxito"); } @PutMapping(path =("/{id}") public ResponseEntity<String> actualizar(@PathVariable("id") Integer id, PersonaTo personaOrigen = this.personaService.buscarPorId(id); personaOrigen.setApellido(cto.getApellido()); personaOrigen.setNombre(cto.getNombre()); this.personaService.actualizar(personaOrigen); return ResponseEntity.ok("Persona actualizada con éxito"); } </pre> <pre> @Path("/personas") public class PersonaControllerImpl { @Inject private IPersonaService personaService; @GET @Path("/{id}") public Response consultar(@PathParam("id") Integer id) { return Response.ok(this.personaService.buscarPorId(id)).build(); } @POST public Response insertar(PersonaTo cto) { this.personaService.insertar(cto); return Response.ok("Persona insertada con éxito").build(); } @PUT @Path("/{id}") public Response actualizar(@PathParam("id") Integer id, PersonaTo cto) { PersonaTo personaOrigen = this.personaService.buscarPorId(id); personaOrigen.setApellido(cto.getApellido()); personaOrigen.setNombre(cto.getNombre()); this.personaService.actualizar(personaOrigen); return Response.ok("Persona actualizada con éxito").build(); } </pre>

Drop Time Native	 <pre>l, cdi, hibernate-orm, jdbc-oracle, narayana-jta, resteasy, resteasy-jsonb, smallrye-context-propagation, vertx] ^C2022-07-07 17:58:48,746 INFO [io.quarkus] (Shutdown thread) sri-demo-quar kus stopped in 0.015s [root@localhost sri-demo-quarkus]#</pre> 
Performance JVM	 
Stress Test	 10 000 Request en 10 s 



Vue.JS

Ejecutada mediante una comparativa entre Vue.JS y React, misma que fue socializada mediante presentación al equipo de Planificación TI:

Dicha comparación se realizó en los siguientes criterios:

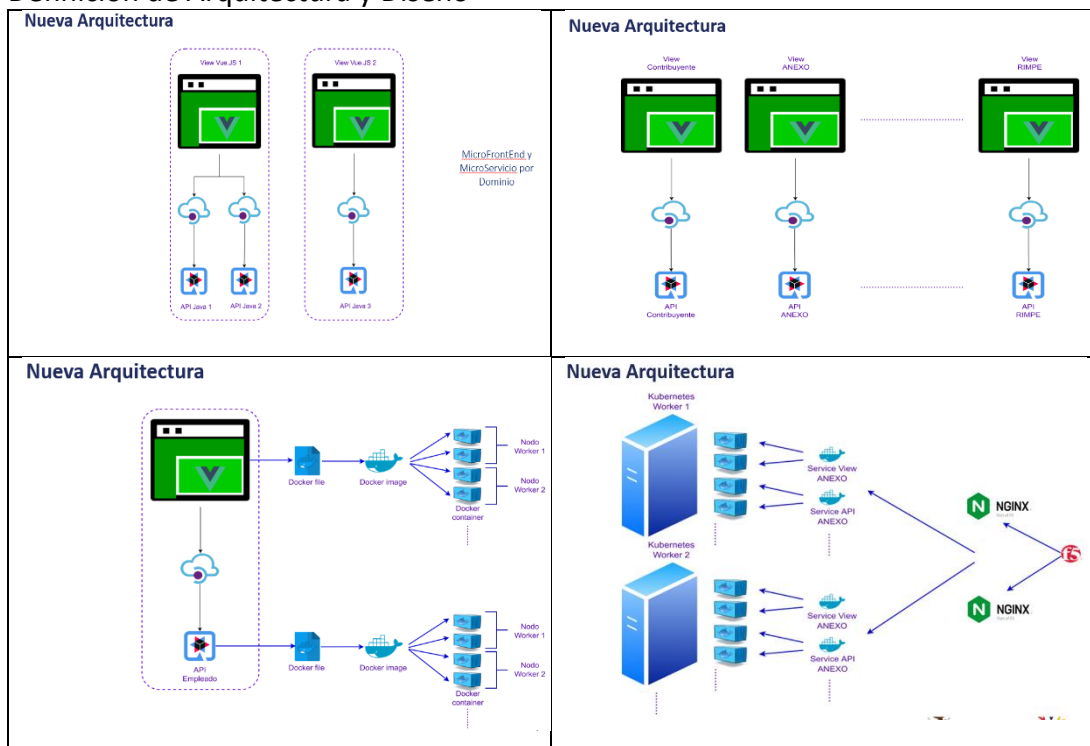
Estructura del proyecto	- Para 1 componente en <u>react</u> usamos 2 archivos, en <u>vye</u> solo 1. - Se tiene varios archivos <u>css</u>, en <u>vye</u> ninguno, lo que son estilos están dentro mismo archivo. <u>Vue</u>.
Componentes	

Paso de datos	 <pre> data() { return { list: [{ 'todo': 'clean the house' }, { 'todo': 'buy milk' }], } }, </pre>  <pre> constructor(props) { super(props); this.state = { // this is where the data goes list: [{ 'todo': 'clean the house' }, { 'todo': 'buy milk' }], }; } </pre> • Para 1 componente en <u>react</u> usamos 2 archivos, en <u>vue</u> solo 1. • Se tiene varios archivos <u>css</u>, en <u>vue</u> ninguno, lo que son estilos están dentro mismo archivo. <u>Vue</u>.
Manejo de listas	 <pre> createNewToDoItem = () => { this .setState(({ lista, tareas }) => ({ lista: [... lista, { tareas }], tareas: '' })); }; </pre> <pre> <input type="text" value={ this .state.todo} onChange={ this .handleInput}/> </pre> <pre> handleInput = e => { this .setState({ todo: e.target.value }); }; </pre>  <pre> createNewToDoItem() { this .list.push({ 'todo': this .todo }); este .todo = ''; } </pre> <pre> <input type="text" v-model="todo"/> </pre>  
Dev Tools	 <pre> 7 requests 443 kB transferred 1.8 MB resources Finish: 1.35 s DOMContentLoaded: 713 ms Load: 984 ms </pre> • Tamaño de proyecto: 126 MB • Tamaño de compilado para producción: 740 Kb  <pre> 6 requests 1.7 MB transferred 1.7 MB resources Finish: 659 ms DOMContentLoaded: 387 ms Load: 644 ms </pre> • Tamaño de proyecto: 128 MB • Tamaño de compilado para producción: 492 KB

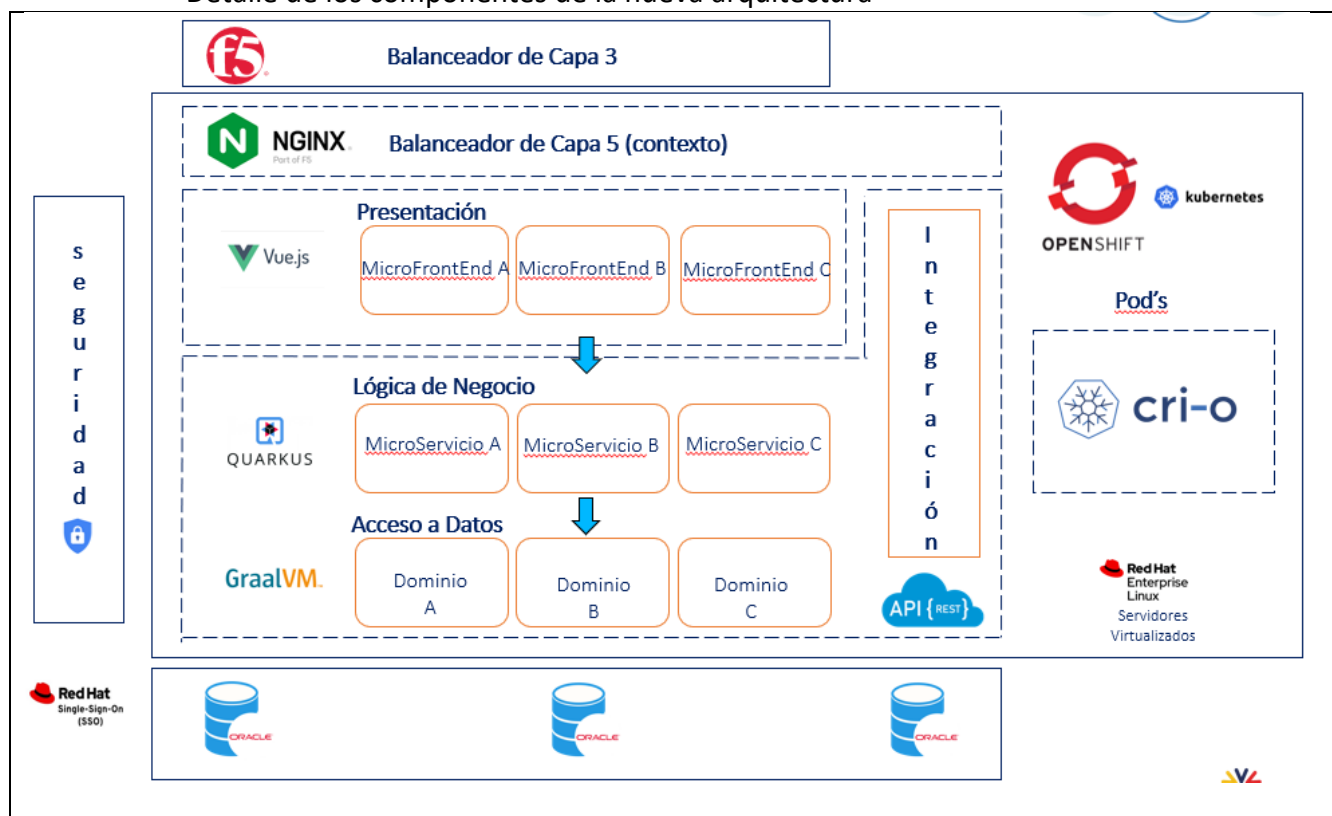
Madurez/ Curva de aprendizaje/ Rendimiento	 	
	REACT	VUE
	Madurez - Lanzamiento en el 2013 - Creado por Facebook - En el mundo el interés por React o Vue es similar - En satisfacción en el uso de react está en 3er lugar. - En cuanto a si volvería a usar o no se encuentra en el primer lugar de si volvería a usar.	- Lanzamiento en el 2014 - Creado por Vue core team - En el mundo el interés por React o Vue es similar - En satisfacción en el uso de Vue está en 4to lugar. - En cuanto a si volvería a usar o no se encuentra en tercer lugar de si volvería a usar.
	Curva de aprendizaje	Medio / Alto
as Internas	Rendimiento De manera general en proyectos de mediana complejidad el rendimiento es el mismo, ya que hay temas donde el uno supera al otro, logrando que se equipare el rendimiento. por ejemplo en la carga inicial, o en el consumo de memoria, o el refrescamiento de DOM. Sin embargo, teóricamente React muestra su fortaleza en rendimiento en aplicaciones de mayor complejidad. - El tiempo de ejecución (duración de las operaciones) de una aplicativo desarrollado con React es menor. - Mejor rendimiento en carga inicial. - Al modificar el estado de un componente, se afecta solo lo necesario y no todo el subárbol de componentes, es decir, menor tiempo de rendimiento. - La asignación de memoria para un aplicativo Vue es mucho menor a uno desarrollado con React.	

3.ARQUITECTURA & DISEÑO:

- Definición de Arquitectura y Diseño

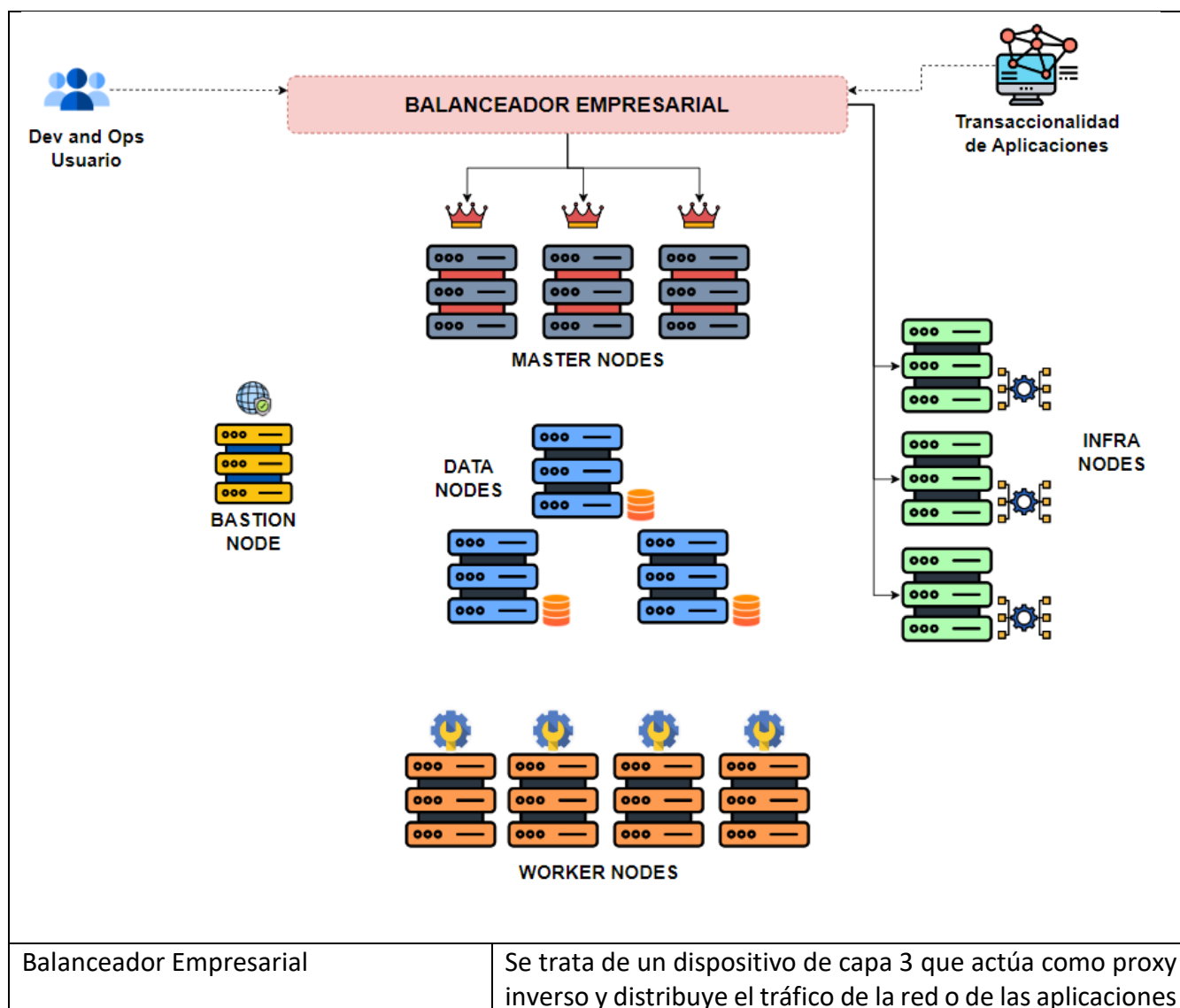


- Detalle de los componentes de la nueva arquitectura



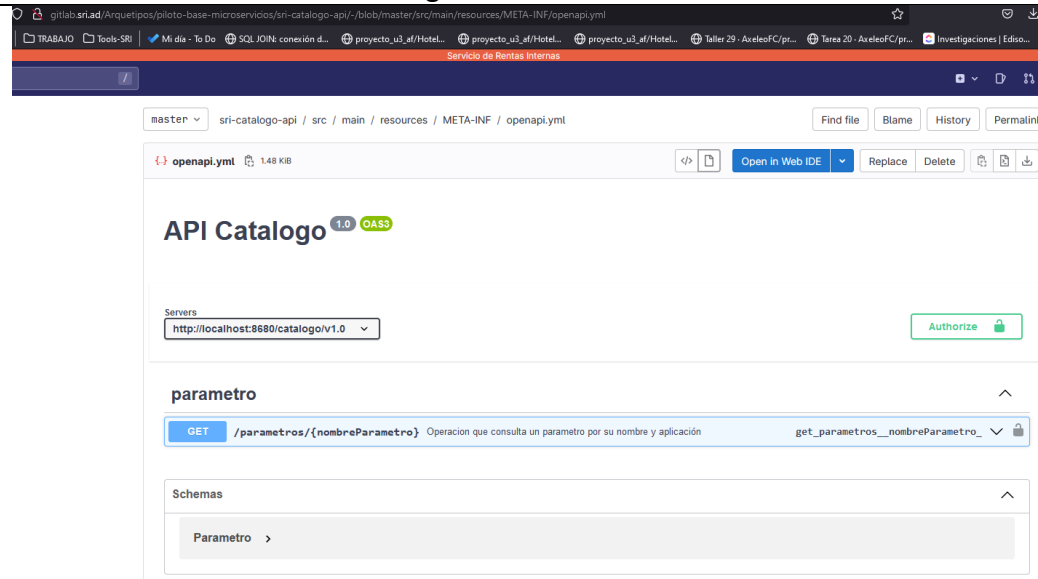
Balanceador de Capa 3	Es un dispositivo que actúa como proxy inverso y distribuye el tráfico de la red o de las aplicaciones a través de varios servidores, lo que aumenta la capacidad (usuarios concurrentes) y la fiabilidad de las aplicaciones.
Balanceador de Capa 5 (contexto)	Permite balancear la carga de los servicios o aplicaciones desplegadas en el clúster de contenedores.
Vue.js	Framework JavaScript para front-end basado en el patrón de diseño model-view-viewmodel que permite crear interfaces de usuario y aplicaciones Single Page Applications (SPA)
Quarkus	Quarkus es un full-stack Java framework nativo para Kubernetes open source integral adaptado para GraalVM y OpenJDK HotSpot. Desarrollado para máquinas virtuales de Java (JVMs) y compilaciones nativas, permite optimizar esta plataforma con especial atención a los contenedores y los entornos serverless, en la nube y Kubernetes on premise. Quarkus utiliza las bibliotecas y los estándares principales de Java, como Eclipse MicroProfile y Spring. Pero también Apache Kafka, RESTEasy (JAX-RS), Hibernate ORM (JPA), Infinispan, Camel y muchos más.
GraalVM	Es una máquina Virtual Java que es capaz de ejecutar código de diversos lenguajes de programación.
Integración RHSSO	Componente de autenticación para FrontEnd (Vue.JS) y para BackEnd (API's RestFul)

Integración API RestFul	Es la capa que se base en servicios de tipo Restful mediante la cual se integran todos los microservicios desplegados en la plataforma de contenedores.
OpenShift	Es una plataforma de desarrollo, con características de Cloud Computing (PaaS) que ofrece la empresa de Red Hat. Esta plataforma se desarrolla a partir de contenedores kubernetes que los desarrolladores utilizan para desplegar aplicaciones en diferentes lenguajes de programación.
CRI-O	Es una implementación de la Container Runtime Interface (CRI) para Kubernetes, que utiliza instancias y entornos en tiempo de ejecución (runtimes) de Open Container Initiative (OCI). La empresa Red Hat inició el proyecto en 2016 y se lo cedió a la Cloud Native Computing Foundation (CNCF) en la primavera de 2019.



	a través de varios servidores, lo que aumenta la capacidad (usuarios concurrentes) y la fiabilidad de las aplicaciones.
Master Nodes	Son los nodos encargados de la administración del cluster permitiendo mantenerlo seguro y además se encarga del monitoreo de los pods desplegados.
Data Nodes	Son nodos de persistencia de datos que permiten mantener la consistencia y estabilidad de la aplicación y datos cuando los contenedores se restauran y eliminan del ambiente.
Worker Nodes	Nodos para atender las cargas de trabajo (PODs), es el lugar donde se despliegan los contenedores con las aplicaciones.
Infra Nodes	Almacenan y proveen funcionalidades de rutas, registro de imágenes, métricas, logging y monitoreo.
Bastion Node	El host Bastion limita el acceso externo a los servidores internos al garantizar que todo el tráfico SSH pase a través del host Bastion.

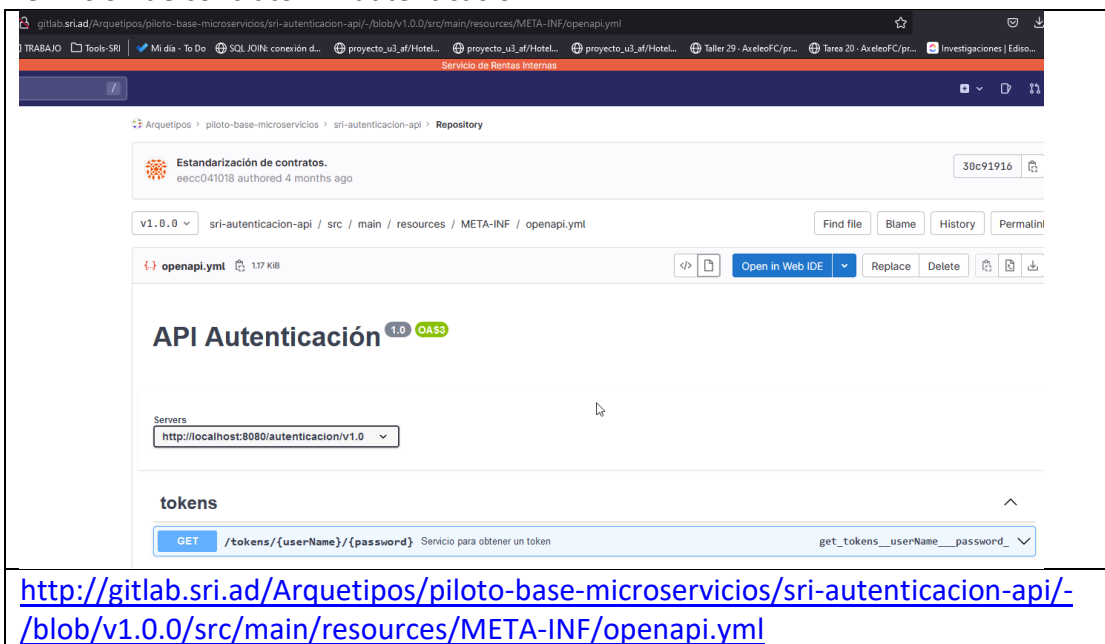
- Definición de contrato API catalogo:



The screenshot shows the GitLab web IDE interface for the file `openapi.yml` located at `sri-catalogo-api / src / main / resources / META-INF / openapi.yml`. The file is 1.48 KB. The preview shows an OpenAPI 3.0 specification for 'API Catalogo'. It includes a 'Servers' section with a base URL, a 'parametro' section with a GET endpoint `/parametros/{nombreParametro}`, and a 'Schemas' section with a 'Parametro' schema.

<http://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-catalogo-api/-/blob/master/src/main/resources/META-INF/openapi.yml>

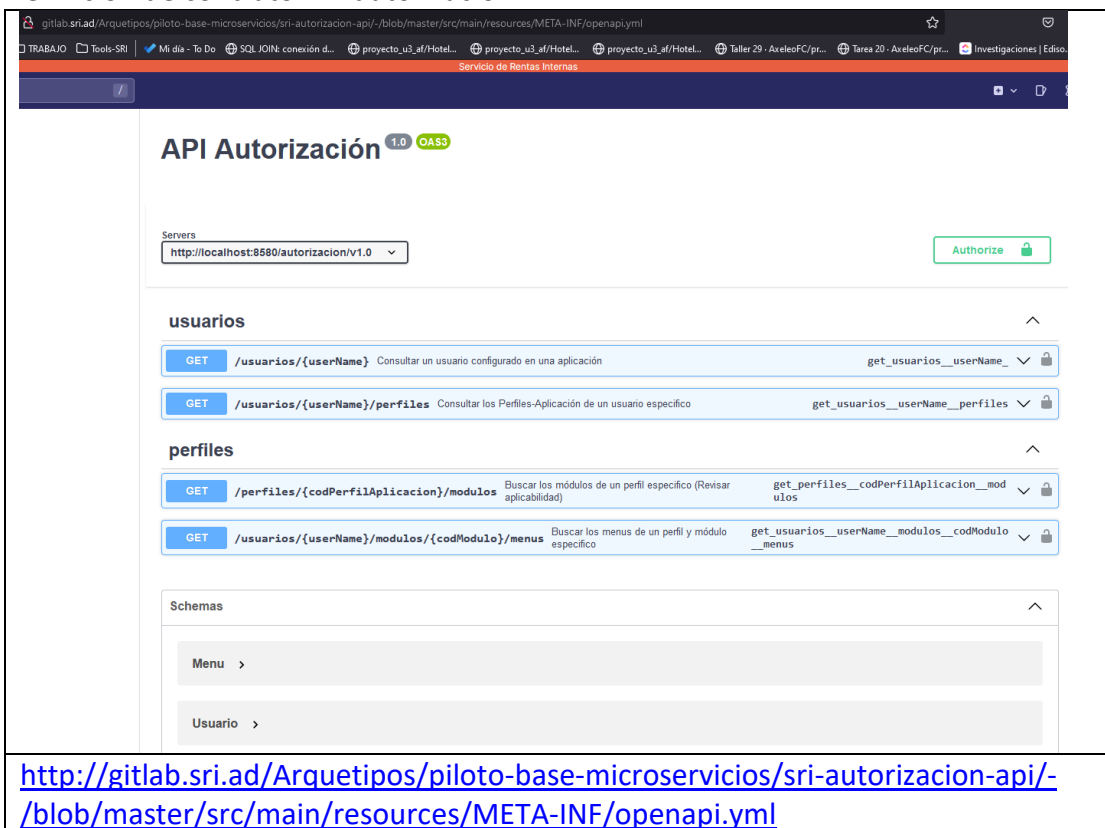
- Definición de contrato API autenticación:



The screenshot shows the GitLab interface for the API Authentication contract. The breadcrumb trail is: Arquetipos > piloto-base-microservicios > sri-autenticacion-api > Repository. The contract is titled "Estandarización de contratos." and was authored 4 months ago. The version is v1.0.0. The path is sri-autenticacion-api / src / main / resources / META-INF / openapi.yml. The contract is titled "API Autenticación" with a 1.0 OAS3 version. The servers section shows a single server: http://localhost:8080/autenticacion/v1.0. The tokens section shows a GET endpoint: /tokens/{userName}/{password} with a description "Servicio para obtener un token" and a response schema get_tokens__userName__password_.

<http://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-autenticacion-api/-/blob/v1.0.0/src/main/resources/META-INF/openapi.yml>

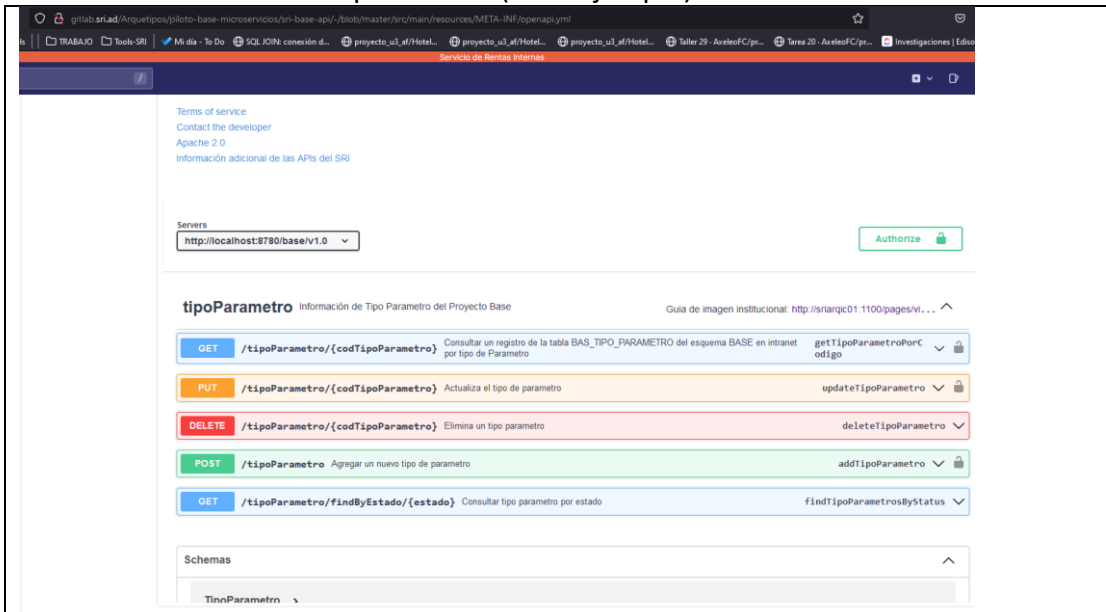
- Definición de contrato API autorización:



The screenshot shows the GitLab interface for the API Authorization contract. The breadcrumb trail is: gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-autorizacion-api/-/blob/master/src/main/resources/META-INF/openapi.yml. The contract is titled "API Autorización" with a 1.0 OAS3 version. The servers section shows a single server: http://localhost:8580/autorizacion/v1.0. There is an "Authorize" button. The usuarios section shows two GET endpoints: /usuarios/{userName} (Consultar un usuario configurado en una aplicación) and /usuarios/{userName}/perfiles (Consultar los Perfiles-Aplicación de un usuario específico). The perfiles section shows two GET endpoints: /perfiles/{codPerfilAplicacion}/modulos (Buscar los módulos de un perfil específico (Revisar aplicabilidad)) and /usuarios/{userName}/modulos/{codModulo}/menus (Buscar los menus de un perfil y módulo específico). The Schemas section shows two schemas: Menu and Usuario.

<http://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-autorizacion-api/-/blob/master/src/main/resources/META-INF/openapi.yml>

- Definición de contrato API parámetro (Base ejemplo):



The screenshot shows the OpenAPI definition for the 'tipoParametro' API. It includes a 'Servers' section with a base URL 'http://localhost:8780/base/v1.0'. The API is titled 'tipoParametro' with a description 'Información de Tipo Parametro del Proyecto Base'. It lists several endpoints:

- GET** /tipoParametro/{codTipoParametro}: Consultar un registro de la tabla BAS_TIPO_PARAMETRO del esquema BASE en intranet por tipo de Parametro. Operation: `getTipoParametroPorCodigo`.
- PUT** /tipoParametro/{codTipoParametro}: Actualiza el tipo de parametro. Operation: `updateTipoParametro`.
- DELETE** /tipoParametro/{codTipoParametro}: Elimina un tipo parametro. Operation: `deleteTipoParametro`.
- POST** /tipoParametro: Agregar un nuevo tipo de parametro. Operation: `addTipoParametro`.
- GET** /tipoParametro/findByEstado/{estado}: Consultar tipo parametro por estado. Operation: `findTipoParametrosByStatus`.

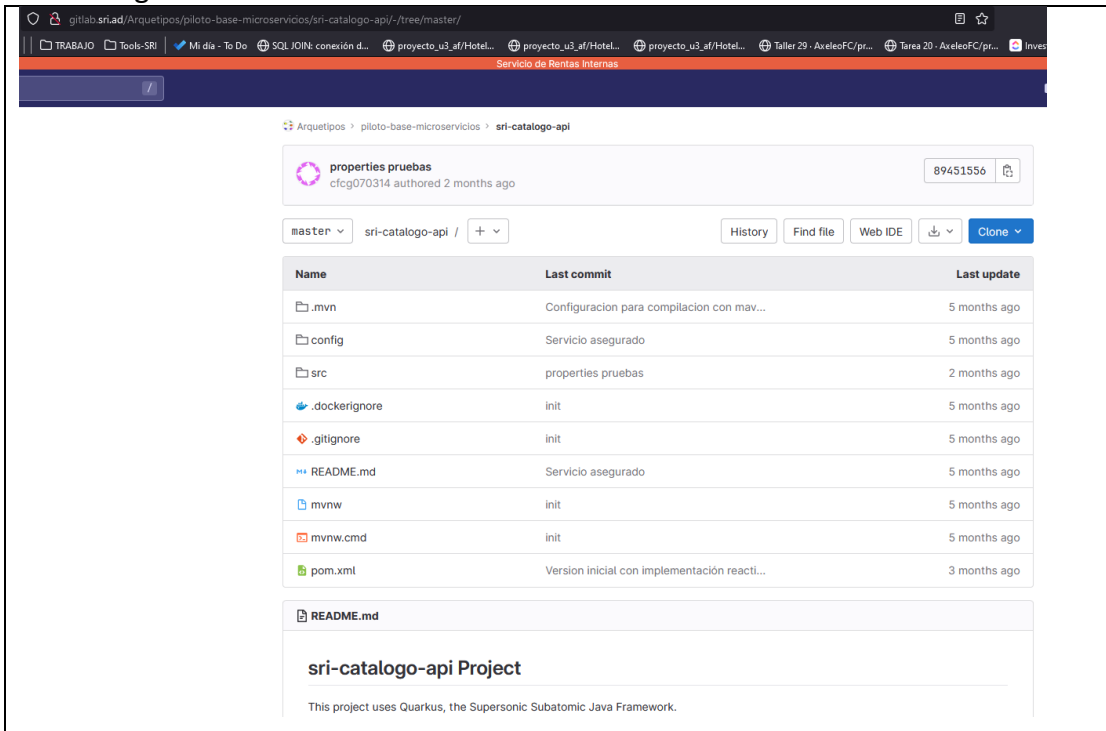
Below the endpoints, there is a 'Schemas' section showing a reference to 'TipoParametro'.

<http://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-base-api/-/blob/master/src/main/resources/META-INF/openapi.yml>

4. DEVELOPMENT BE:

Implementación componentes Back End:

- API catalogo:



The screenshot shows the GitLab repository page for the 'sri-catalogo-api' project. The repository is owned by 'properties pruebas' and was authored 2 months ago. It has 89451556 commits. The repository is located at 'gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-catalogo-api'.

The repository contains the following files and folders:

Name	Last commit	Last update
.mvn	Configuracion para compilacion con mav...	5 months ago
config	Servicio asegurado	5 months ago
src	properties pruebas	2 months ago
.dockerignore	init	5 months ago
.gitignore	init	5 months ago
README.md	Servicio asegurado	5 months ago
mvnw	init	5 months ago
mvnw.cmd	init	5 months ago
pom.xml	Version inicial con implementación reacti...	3 months ago

The README.md file contains the following text:

sri-catalogo-api Project

This project uses Quarkus, the Supersonic Subatomic Java Framework.

<http://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-catalogo-api/-/tree/master/>

- API autenticación:

gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-autenticacion-api/-/tree/v1.0.0

Arquetipos > piloto-base-microservicios > sri-autenticacion-api > Repository

use Estandarización de contratos.
eccc041018 authored 4 months ago

v1.0.0 sri-autenticacion-api / +

Name	Last commit	Last update
.mvn/wrapper		5 months ago
src	Estandarización de contratos.	4 months ago
.dockerignore		5 months ago
.gitignore		5 months ago
README.md		5 months ago
mvnw		5 months ago
mvnw.cmd		5 months ago
pom.xml		5 months ago

README.md

sri-autenticacion-api Project

<http://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-autenticacion-api/-/tree/v1.0.0>

- API autorización:

gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-autorizacion-api/-/tree/master/

Arquetipos > piloto-base-microservicios > sri-autorizacion-api

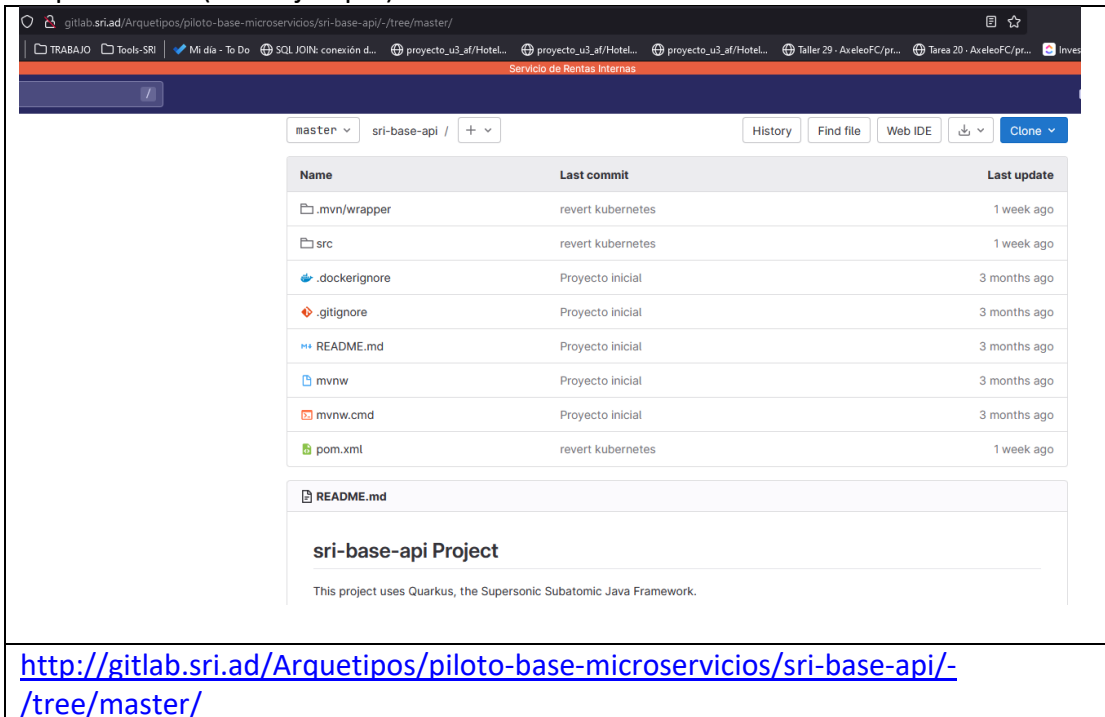
Se modifica la autorización de permit por authenticated
Munoz Ontaneda, Carmen Estefania authored 2 weeks ago

master sri-autorizacion-api / +

Name	Last commit	Last update
.mvn	Configuraciones para compilacion con m...	5 months ago
src	Se modifica la autorización de permit por...	2 weeks ago
.dockerignore	Estructura inicial del proyecto Quarkus	6 months ago
.gitignore	Estructura inicial del proyecto Quarkus	6 months ago
mvnw	Estructura inicial del proyecto Quarkus	6 months ago
mvnw.cmd	Estructura inicial del proyecto Quarkus	6 months ago
pom.xml	configuraciones	4 months ago

<http://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-autorizacion-api/-/tree/master/>

- API parámetro (Base ejemplo):



The screenshot shows the GitLab web interface for the repository 'sri-base-api' at the 'master' branch. The page displays a list of files and their commit history. Below the file list, the 'README.md' file is expanded, showing the project name 'sri-base-api Project' and a description: 'This project uses Quarkus, the Supersonic Subatomic Java Framework.'

Name	Last commit	Last update
.mvn/wrapper	revert kubernetes	1 week ago
src	revert kubernetes	1 week ago
.dockerignore	Proyecto inicial	3 months ago
.gitignore	Proyecto inicial	3 months ago
README.md	Proyecto inicial	3 months ago
mvnw	Proyecto inicial	3 months ago
mvnw.cmd	Proyecto inicial	3 months ago
pom.xml	revert kubernetes	1 week ago

sri-base-api Project

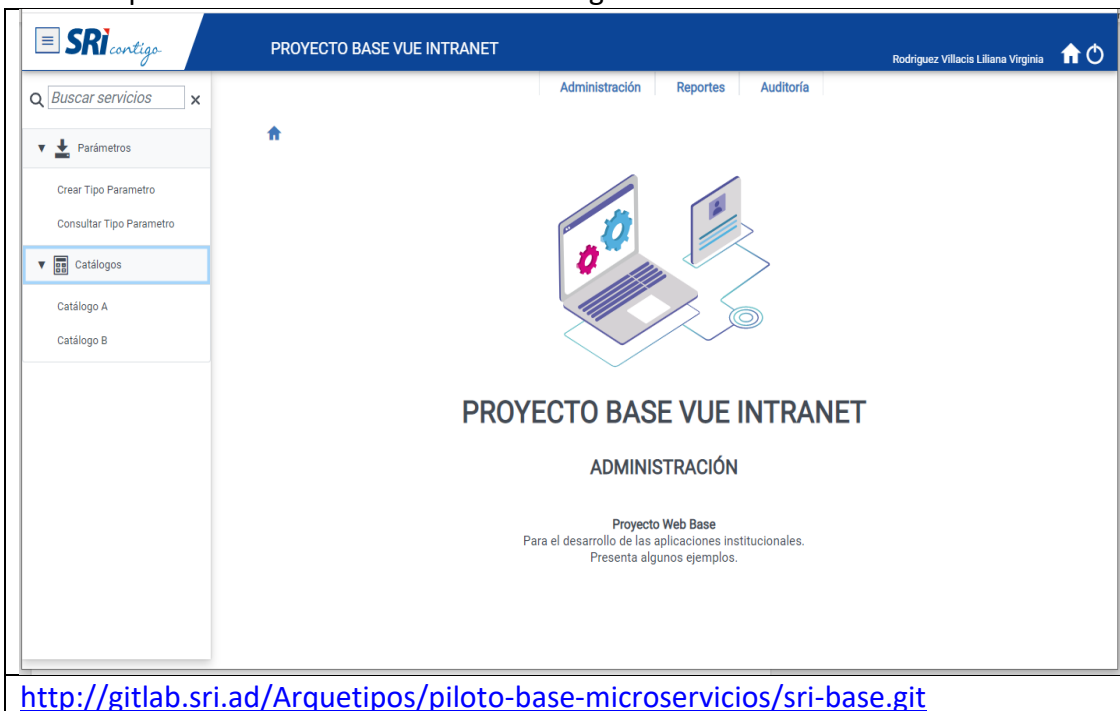
This project uses Quarkus, the Supersonic Subatomic Java Framework.

<http://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-base-api/-/tree/master/>

5. DEVELOPMENT FE:

Implementación componente Front End:

- Vista template base conforme la Guía de Imagen Institucional:



The screenshot shows the 'PROYECTO BASE VUE INTRANET' web application. The interface includes a header with the 'SRI contigo' logo, the title 'PROYECTO BASE VUE INTRANET', and the user name 'Rodríguez Villacis Liliana Virginia'. The main content area features a sidebar with navigation options: 'Parámetros' (with sub-options 'Crear Tipo Parametro' and 'Consultar Tipo Parametro') and 'Catálogos' (with sub-options 'Catálogo A' and 'Catálogo B'). The central area displays a large illustration of a laptop and a tablet connected by a network line, with the text 'PROYECTO BASE VUE INTRANET ADMINISTRACIÓN' and a description: 'Proyecto Web Base Para el desarrollo de las aplicaciones institucionales. Presenta algunos ejemplos.'

<http://gitlab.sri.ad/Arquetipos/piloto-base-microservicios/sri-base.git>

- Seguridad páginas web:

```
function initKeycloak() {
  const keycloak = new Keycloak({
    url: constantesAplicacion.SERVIDOR_SSO,
    realm: constantesAplicacion.REALM,
    clientId: constantesAplicacion.CIENTE_SSO,
    'enable-cors': true
  });

  keycloak.init({ onLoad: 'login-required', promiseType: 'native', checkloginiframe: false }).success(authenticated => {

    const app = createApp(App).use(i18n)

    app.use(createPinia())
    .use(router)
    .use(Vuex)
    .use(Pinia)
    .component("fa", FontAwesomeIcon)
    .component("InputText", InputText)
    .component("Message", Message)
    .component("Calendar", Calendar)
    .component("Button", Button)
    .component("DataTable", DataTable)
    .component("Column", Column)
    .component("RadioButton", RadioButton)
    .component("InlineMessage", InlineMessage)
    .component("Dialog", Dialog)
    .component("Textarea", Textarea)
    .directive('tooltip', Tooltip)
    obtenerUsername(keycloak)

    app.mount('#app')
  }).error(function() {
    console.error("Error al inicializar SSO");
    return false
  });
}

window.onload = initKeycloak();

const obtenerUsername = (keycloak) =>{
  const (preferred_username) = jwt_decode(keycloak.token)
  const usuarioStore = useUsuarioStore()
  usuarioStore.setInformacionUsuario(preferred_username, keycloak.token, keycloak)
}
```



7. PREPARACIÓN AMBIENTE:

Instalación de ambientes:

- Cluster Kubernetes:

Nodo master activo

srikubermaster

```
[adminpti@srikubermaster ~]$ kubectl get nodes
NAME                 STATUS    ROLES    AGE   VERSION
srikubermaster.sri.ad Ready    control-plane   152d   v1.25.1
srikubernet01.sri.ad Ready    <none>         151d   v1.25.1
srikubernet02.sri.ad Ready    <none>         151d   v1.25.1

[adminpti@srikubermaster ~]$ kubectl get svc
NAME                 TYPE        CLUSTER-IP    EXTERNAL-IP  PORT(S)          AGE
kubernetes           ClusterIP   10.96.0.1     <none>       443/TCP          152d
sri-autorizacion-api-jvm-pti NodePort    10.98.76.137  <none>       80:30001/TCP     48d
sri-base-api-pti     NodePort    10.111.49.115 <none>       80:30000/TCP     14d
```

Nodo worker 1 activo

srikubernet01

```
[adminpti@srikubernet01 ~]$ sudo systemctl status kubelet --no-pager
● kubelet.service - kubelet: The Kubernetes Node Agent
   Loaded: loaded (/usr/lib/systemd/system/kubelet.service; enabled; vendor preset: disabled)
   Drop-In: /usr/lib/systemd/system/kubelet.service.d
            └─10-kubeadm.conf
   Active: active (running) since Thu 2023-01-19 10:45:35 -05; 3 weeks 5 days ago
     Docs: https://kubernetes.io/docs/
   Main PID: 570904 (kubelet)
    Tasks: 19 (limit: 74952)
   Memory: 75.7M
   CGroup: /system.slice/kubelet.service
            └─570904 /usr/bin/kubelet --bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf --kubeconfig=/etc/kubernetes/kubelet.conf
```

Nodo worker 1 activo

srikubernet02

```
[adminpti@srikubernet02 ~]$ sudo systemctl status kubelet --no-pager
● kubelet.service - kubelet: The Kubernetes Node Agent
   Loaded: loaded (/usr/lib/systemd/system/kubelet.service; enabled; vendor preset: disabled)
   Drop-In: /usr/lib/systemd/system/kubelet.service.d
            └─10-kubeadm.conf
   Active: active (running) since Thu 2023-01-19 05:45:32 -05; 3 weeks 5 days ago
     Docs: https://kubernetes.io/docs/
   Main PID: 568273 (kubelet)
    Tasks: 20 (limit: 74952)
   Memory: 69.7M
   CGroup: /system.slice/kubelet.service
            └─568273 /usr/bin/kubelet --bootstrap-kubeconfig=/etc/kubernetes/bootstrap-kubelet.conf --kubeconfig=/etc/kubernetes/kubelet.conf
```

- Registry privado:

Servicio nexus activo

```
[pti@sribpms ~]$ systemctl status nexus.service --no-pager
● nexus.service - Nexus Service
   Loaded: loaded (/etc/systemd/system/nexus.service; enabled; vendor preset: disabled)
   Active: active (running) since Sat 2022-10-29 19:35:55 -05; 3 months 17 days ago
     Main PID: 1370 (java)
        Tasks: 120 (limit: 102125)
       Memory: 2.0G
      CGroup: /system.slice/nexus.service
              └─1370 /usr/java/latest/bin/java -server -Dinstall4j.jvmDir=/usr/java/latest -Dext4j.moduleName=/var/APP/nexus-oss/nexus/bin/nexus -XX:+UnlockDiag...

Warning: Journal has been rotated since unit was started. Log output is incomplete or unavailable.
[pti@sribpms ~]$
```

Puerto de consola web activo

```
[pti@sribpms ~]$ netstat -tunlp | grep 9081
(Not all processes could be identified, non-owned process info
will not be shown, you would have to be root to see it all.)
tcp        0      0 0.0.0.0:9081          0.0.0.0:*            LISTEN      -
```

- GitLab:

Servicio GitLab activo

```
[adminpti@sriptigitlab ~]$ sudo gitlab-ctl status
run: alertmanager: (pid 2570) 2748939s; run: log: (pid 2564) 2748939s
run: gitaly: (pid 2560) 2748939s; run: log: (pid 2559) 2748939s
run: gitlab-exporter: (pid 2558) 2748939s; run: log: (pid 2557) 2748939s
run: gitlab-kas: (pid 2553) 2748939s; run: log: (pid 2551) 2748939s
run: gitlab-workhorse: (pid 2567) 2748939s; run: log: (pid 2550) 2748939s
run: logrotate: (pid 3077399) 2099s; run: log: (pid 2552) 2748939s
run: nginx: (pid 2540) 2748939s; run: log: (pid 2537) 2748939s
run: node-exporter: (pid 2566) 2748939s; run: log: (pid 2547) 2748939s
run: postgres-exporter: (pid 2563) 2748939s; run: log: (pid 2561) 2748939s
run: postgresql: (pid 2539) 2748939s; run: log: (pid 2536) 2748939s
run: prometheus: (pid 3095692) 0s; run: log: (pid 2555) 2748939s
run: puma: (pid 2565) 2748939s; run: log: (pid 2549) 2748939s
run: redis: (pid 2562) 2748939s; run: log: (pid 2548) 2748939s
run: redis-exporter: (pid 2546) 2748939s; run: log: (pid 2544) 2748939s
run: sidekiq: (pid 2541) 2748939s; run: log: (pid 2538) 2748939s
[adminpti@sriptigitlab ~]$
```

8. DESPLIEGUE:

- Despliegue de aplicación piloto:

Pods desplegados en el cluster

```
[adminpti@srikubermaster ~]$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
ghost	1/1	Running	1	62d
sri-autorizacion-api-jvm-pti-5b9897bcb7-krrc5	1/1	Running	1	49d
sri-base-api-pti-8d6d58d8c-kln4f	0/1	ErrImagePull	0	14d
vault-csi-provider-bl5qf	1/1	Running	1	77d
vault-csi-provider-gcrgf	1/1	Running	1	77d

```
[adminpti@srikubermaster ~]$
```

7. Conclusiones

- Para la implementación del Back End de las aplicaciones con la arquitectura resiliente se usó Quarkus que proporciona un mejor rendimiento vs. Spring Boot.
- El uso de tecnologías Nativas (Quarkus Native) proporciona un mejor aprovechamiento de la Java Virtual Machine.
- Vue.js como implementación del Front End proporciona una curva de aprendizaje más corta en relación con React.js y Angular.
- La forma más adecuada de definir la construcción de microservicios es usando el enfoque de desarrollo Domain Driven Design (DDD).
- El runtime que permite la orquestación de contenedores es Kubernetes, mismo que fue adoptado como plataforma de contenerización para la nueva arquitectura resiliente.
- El runtime de contenedores que proporciona una interacción nativa con Kubernetes es Cri-o.
- El uso de software Open Source (Kubernetes) demanda realizar actividades complementarias para configurar un clúster de contenedores, mismas que se detallan a continuación:
 1. El balanceador de capa 5 como Endpoint de páginas web y APIs.
 2. Manejo de credenciales para cadenas sensibles.
 3. Configuración de nodos workers como miembros del cluster.
 4. Configuración manual de Manifiestos
- Kubernetes como solución Open Source no integra prestaciones que una plataforma licenciada si las considera, como son:
 1. Consola de administración y monitoreo.
 2. Parchado de vulnerabilidades que garanticen compatibilidad de todos los componentes del cluster y que sean automáticos.
 3. Soporte del fabricante.
 4. Integración con un Registry de confianza que garantice imágenes seguras.

8. Recomendaciones

- Se recomienda continuar con la implementación de la fase dos de la nueva arquitectura resiliente que involucra la adopción de un flujo DEVOPS que incluya CI/CD.
- Se recomienda elaborar la documentación que permita estandarizar la implementación de aplicaciones basadas en la nueva arquitectura resiliente.
- Se recomienda utilizar para entornos productivos de misión crítica un clúster de contenerización suscrito con un fabricante que brinde el soporte de esta.

- Se recomienda ejecutar una iniciativa que permita trabajar el cambio cultural que demanda la adopción de estas nuevas tecnologías y arquitectura resiliente con el objetivo de que en la implementación exista la menor resistencia al cambio.
- Se recomienda que la fase inicial de la implementación de la nueva arquitectura se realice con el acompañamiento de un especialista (fabricante) que cuente con la experiencia en la implementación de arquitecturas resilientes.

9. Glosario

Arquitectura resiliente: Se lo debe entender como la capacidad que tiene un sistema, -en este caso la arquitectura- para continuar brindando altos niveles de calidad de servicio aun cuando una parte de éste se encuentre fuera de servicio temporal o permanentemente.

CI/CD: Integración continua (CI) y despliegue continuo (CD).

Cluster de Kubernetes: Los clústeres son generalmente un grupo de ordenadores que funcionan como si fueran uno. En Kubernetes, en lugar de ordenadores físicos, se combinan varios nodos en clústeres. Estos nodos son máquinas virtuales o físicas.

Dependency hell: Infierno de dependencia, hace referencia a problemas de dependencias de aplicaciones de software.

DEVELOPMENT BE: Desarrollo del back end (back end es la parte de un sistema que procesa la entrada desde el front end).

DEVELOPMENT FE: Desarrollo del front end (front end es la parte del software que interactúa con los usuarios).

DevOps: es un marco de trabajo y una filosofía en constante evolución que promueve un mejor desarrollo de aplicaciones en menos tiempo y la rápida publicación de nuevas o revisadas funciones de software o productos para los clientes.

Down Time: Hace referencia a períodos en los que un sistema no está disponible.

Java EE: Java Platform Enterprise Edition o Java EE es una plataforma de programación parte de la Plataforma Java para desarrollar y ejecutar software de aplicaciones en el lenguaje de programación Java.

JBOSS EAP: Red Hat JBoss Enterprise Application Platform, servidor de aplicaciones open source compatible con Java EE 8 que le permite diseñar aplicaciones Java una sola vez e implementarlas en cualquier entorno.

Product backlog: El product backlog (o pila de producto) es un listado de todas las tareas que se pretenden hacer durante el desarrollo de un proyecto.

Quarkus: Es un marco integral de Java diseñado para su implementación en Kubernetes y creado para las compilaciones propias y las máquinas virtuales Java (JVM). Permite optimizar Java especialmente para los contenedores y los entornos sin servidor, de nube y de Kubernetes.

Vue: Es un framework de JavaScript de código abierto para la construcción de interfaces de usuario y aplicaciones de una sola página.